DR. VISHWANATH KARAD MIT WORLD PEACE UNIVERSITY, PUNE
Department of Computer Engineering and Technology | School of Engineering & Technology
Course: Cloud Computing and DevOps (CSE30040) | Faculty: Prof. Pranati Waghodekar

CCA 2 INDIVIDUAL ASSESSMENT SUBMISSION REPORT

Dynamic Sports Management Web Application & Automated CI/CD Pipeline

Field	Submission Details
Student Name	Kapil (Student — B.Tech CSE)
PRN Number	103223XXXX
Roll No. / Panel	Panel A / Roll No. XX
Project Title	ArenaOps — Sports Management System & Automated CI/CD Pipeline
GitHub Repository URL	https://github.com/kapil2908/sports-manager.git
Live Render App URL	https://sports-manager-gth0.onrender.com
Submission Date	September 28, 2026
Subject Faculty	Prof. Pranati Waghodekar

1. Problem Statement & Project Overview

University sports departments manage dozens of concurrent inter-college fixtures across Football, Cricket, Basketball, Badminton, Athletics, and Esports. Manual spreadsheet tracking leads to scheduling conflicts, unvalidated data entry, and zero visibility into live tournament completion rates. Furthermore, traditional manual software deployment introduces configuration drift, untested code regressions, and downtime.

To solve both operational and software delivery challenges, **ArenaOps (Sports Management System)** was built from scratch using **Node.js, Express, and EJS** with a dark responsive UI, paired with a strict three-stage **GitHub Actions CI/CD pipeline**, **Docker (node:22-alpine)** containerization, and automated webhook deployment to **Render**.

Key Technical Capabilities Implemented:

- **Server-Side Rendered Dashboard (GET /):** Displays real-time tournament metrics (total fixtures, completion percentage, live match counter, average intensity rating 1–5, and active 7-char Git SHA).
- **Validated & XSS-Sanitized Fixture Creation (POST /matches):** Enforces required fields, category/status allowlists, numeric rating bounds (1–5), and HTML entity escaping (&, <, >, ", ').
- **Observability & Telemetry Endpoints:** Exposes **GET /health** (status, commit SHA, uptime, ISO timestamp) for container readiness probes and **GET /api/matches** for JSON telemetry.

2. CI/CD Pipeline Architecture & Stage Breakdown

The Continuous Integration and Continuous Deployment workflow (**.github/workflows/ci-cd.yml**) is triggered automatically on every **push** and **pull_request** targeting the **main** branch. It enforces sequential quality gates using the **needs:** directive.

Pipeline Stage	Tools & Commands Executed	Trigger & Safety Gate
Stage 1: Lint & Unit/Integration Tests (test)	Node.js 22, npm ci, ESLint v9 Flat Config (npm run lint), Node:test Runner (npm test)	Runs on push & PR to main. Fails immediately on any syntax/lint error or failed assertion.
Stage 2: Docker Container Build & Smoke Test (build)	docker build --build-arg GIT_SHA=${{ github.sha }}, docker run -d -p 3000:3000, curl -f /health	needs: test. Verifies production Alpine image builds cleanly and serves HTTP 200 on /health.
Stage 3: Continuous Deployment to Render (deploy)	curl -fsS -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}&ref;=${{ github.sha }}"	needs: build. Restricted to push events on refs/heads/main. Keeps Render Auto-Deploy OFF.

Stage 1: Automated Linting & Node:test Suite (Job: test)

Static analysis is executed via ESLint v9 flat configuration (**eslint.config.js**) configured for CommonJS with readonly **fetch** and **URLSearchParams** globals. Next, **test/app.test.js** spins up an isolated Express server on ephemeral port 0 before each test and resets the store via **app.resetStore()**:

- **Test 1:** GET /health returns HTTP 200 and { status: 'ok', commit, timestamp }.
- **Test 2:** POST /matches with valid input returns HTTP 201 and creates a new entry (verified via GET /api/matches).
- **Test 3:** POST /matches with missing required field returns HTTP 400 AND with invalid numeric value returns HTTP 400.
- **Test 4:** GET /api/matches returns valid JSON with contracts/matches array and stats object with numeric fields.

Stage 2: Docker Image Build, GIT_SHA Injection & Smoke Test (Job: build)

Using **node:22-alpine**, the Dockerfile installs production-only dependencies (**npm ci --omit=dev**), accepts **ARG GIT_SHA=local**, and exposes **ENV GIT_SHA=$GIT_SHA** so the running app displays the exact 7-character commit hash in the UI footer and **/health** JSON. The container runs under non-root **USER node** and is smoke-tested inside GitHub Actions via **curl -f http://localhost:3000/health** before teardown.

Stage 3: Zero-Downtime Render Webhook Trigger (Job: deploy)

Only when both **test** and **build** succeed on a push to **main** does the pipeline invoke the encrypted **RENDER_DEPLOY_HOOK** secret with the exact commit SHA, ensuring unverified code never deploys.

3. Pipeline Gate Verification: Failure Demo & Recovery

To validate that the CI/CD pipeline actively protects production from defective code, a controlled failure simulation was executed and verified against the pipeline dependency graph:

- **Step 1 (Intentional Fault Injection):** An assertion in **test/app.test.js** for **GET /health** was temporarily modified to expect **status: 'broken'** instead of **'ok'**.
- **Step 2 (CI Quality Gate Rejection):** On pushing the commit, Job 1 (■ **Lint & Unit/Integration Tests**) executed **npm test**, detected the **AssertionError [ERR_ASSERTION]**, and exited with code 1.
- **Step 3 (Downstream Jobs Skipped):** Because Job 2 (**build**) specifies **needs: test** and Job 3 (**deploy**) specifies **needs: build**, GitHub Actions automatically skipped both the Docker build and the Render deployment webhook.
- **Step 4 (Production Stability Preserved):** The live Render service remained online serving the previous healthy commit without interruption. Restoring the assertion returned all 3 pipeline jobs to green.

4. Verification Summary & Quality Metrics

Verification Check	Command / Endpoint	Observed Output
ESLint v9 Static Analysis	npm run lint	0 errors, 0 warnings across server.js, app.js, test/app.test.js
Node:test Automated Suite	npm test	4 / 4 tests passing (health, valid POST 201, invalid POST 400, API JSON stats)
Docker Container Smoke Test	curl -f http://localhost:3000/health	HTTP 200 OK — {"status":"ok","commit":"...","uptime":5,"timestamp":"..."}
Git History & Branch Topology	git log --oneline --graph --all	10 structured commits + feature/docker-ci-pipeline --no-ff merge commit

5. Engineering Challenges & Key Learnings

- **Ephemeral Port Allocation in Tests:** Binding to a fixed port during automated tests causes EADDRINUSE conflicts. Using **app.listen(0)** in **beforeEach** dynamically assigns an available OS port for every test.
- **In-Memory State Isolation:** Exporting **app.resetStore()** ensured mutations in POST tests never polluted subsequent GET assertions.
- **Traceable Builds via GIT_SHA:** Passing **--build-arg GIT_SHA=${{ github.sha }}** into Docker bridges CI metadata with runtime observability in the footer and **/health** endpoint.

6. Project Artifacts & Submission Links

Resource / Artifact	Direct URL / Repository Path
GitHub Source Repository	https://github.com/kapil2908/sports-manager.git
Live Render Cloud Deployment	https://sports-manager-gth0.onrender.com
GitHub Actions CI/CD Runs	https://github.com/kapil2908/sports-manager/actions
CI/CD Workflow Definition	.github/workflows/ci-cd.yml

7. Comprehensive Viva Voce Questions & Answers

Q1: What is the difference between Continuous Integration (CI) and Continuous Deployment (CD)?

Answer: Continuous Integration (CI) automatically lints, builds, and tests every code change merged into a shared repository branch to catch integration bugs early. Continuous Deployment (CD) extends CI by automatically containerizing, smoke-testing, and deploying every build that passes all automated tests directly to production without manual intervention.

Q2: How does the 'needs:' keyword enforce pipeline safety in GitHub Actions?

Answer: GitHub Actions runs jobs in parallel by default. Declaring **needs: test** on the **build** job and **needs: build** on the **deploy** job creates a strict directed acyclic graph (DAG). If ESLint or any unit test fails, downstream Docker build and Render deployment jobs are automatically skipped.

Q3: Why do we store the Render Deploy Hook URL in GitHub Secrets instead of hardcoding it?

Answer: A deploy hook URL embeds a sensitive token capable of triggering production deployments at will. Storing it in **secrets.RENDER_DEPLOY_HOOK** keeps the credential encrypted at rest, prevents leaks in public git history, and masks the value in CI runner logs.

Q4: Why containerize the Node.js application using Docker (node:22-alpine)?

Answer: Docker packages the application code, exact Node 22 runtime, and locked production dependencies into an immutable image. Running a detached container smoke test against **/health** as non-root user **node** guarantees identical behavior between CI runners and cloud production.

End of CCA 2 Submission Report — Verified by Automated CI/CD Suite (MIT-WPU CSE30040)